Secure Development Standards

Early Access · MessageFoundry v0.3.2 · July 2026

Document	Secure Development Standards
Applies to	Any application developed under this standard. **MessageFoundry (MEFOR)** is the reference implementation (Appendix A).
Maintained by	Project maintainers (open-source). Each deploying organization assigns its own local owner.
Status	Published — adopter-facing
Version	2.2
Date	July 30, 2026
License	Publishable under the project's open-source license; intended to be shared with adopters and reused across projects.
Review cadence	At least annually, and on any material architecture or threat change
Aligns to	NIST SP 800-218 (SSDF) · NIST SP 800-115 · NIST SP 800-66 Rev. 2 (HIPAA Security Rule) · OWASP ASVS 5.0 Level 3. Its Spec-Driven Development practices (§5) are a distilled synthesis by this document — not an external standard or certification.

1. Purpose and scope

This is the secure development standard for an **open-source software project intended for use in regulated environments, including healthcare** (handling of protected health information, PHI).

It is written to serve three audiences:

1. **The development team** — a consistent engineering bar to build to, across this and future projects.
2. **Deploying organizations** — both the maintainer's own organization and any other organization that adopts the open-source software, who need evidence that it was built and tested securely.
3. **Future projects** — this standard is **project-agnostic**; any new application can be developed under it without rewriting it.

How the standard is structured. The body (§2–§10) states requirements that apply to *any* application built under this standard. Each application records its own specifics — technology stack, applicable verification scope, the interface mechanisms it implements — in a **per-project Applicability Profile**. MEFOR's profile is Appendix A; future projects add their own (Appendix B, C, …).

Companion standards. Two companions extend this baseline. The Secure AI-Assisted Development Standards governs *building with an AI coding assistant* (risk tiers, provenance, guardrails). The Code Quality & Anti-Slop Standards governs *judging whether the resulting code is good, not "AI slop"* — an evidence-based rubric aligned to ISO/IEC 25010. This document states the security baseline both build on; the three are complementary (process → build → outcome).

Open-source note. The software is developed in the open. This standard, and the project's security attestations, are publishable so that adopters can rely on them or extend them for their own environment.

---

2. Shared responsibility

Because the software is built by one party and deployed by others, responsibilities split cleanly. Stating the split prevents either side from assuming the other has it covered.

The is responsible for	The is responsible for
Secure development practices (§4)	Its own environment, host, and network security
Secure-by-default configuration	Identity, credential, and key management in its environment
Security testing and attestation of the software (§6)	Backups, disaster recovery, and availability
Vulnerability response and disclosure (§8)	Its own compliance program — HIPAA Security Rule obligations, risk assessments of the deployment, and Business Associate Agreements where applicable
Documentation and evidence (§9)	Operational monitoring, patching, and incident response

No certification or agreement is conferred by the software itself. A deploying organization operates the software under *its own* programs; an attestation that the software was built securely is evidence for that organization's assessment, not a substitute for it.

---

3. How this maps to NIST (overview)

Three NIST publications cover three different questions. Together they form the standard:

Question	NIST publication	Section
How is the software built? (secure development process)	SP 800-218 (SSDF)	§4
How is it tested? (security testing methodology)	SP 800-115	§6

Question	NIST publication	Section
What controls does it implement? (security/privacy controls + HIPAA safeguards)	SP 800-66 Rev. 2	§7

These are complementary to the **OWASP ASVS 5.0 Level 3** verification (source-assisted and hands-on; performed internally today, with an independent assessment as a best-practice pre-production add-on — ASVS itself does not require independence): ASVS verifies that the built application is secure; SSDF attests to how it was built; 800-66 maps the HIPAA safeguards.

A note on claims and wording (read before publishing any claim)

NIST does **not** issue certificates for these frameworks. The project may **display alignment claims**, but words them honestly and backs them with evidence:

- **Use:** "Built to NIST SP 800-218 (SSDF)," "NIST SSDF–aligned," "tested per NIST SP 800-115," "controls mapped to NIST SP 800-66 Rev. 2," "HIPAA-compliant deployment supported."
- **Do not use:** "NIST certified" or any phrasing implying a certificate exists.
- **Back every claim** with the implemented practice and its evidence (this standard, test reports, the ASVS attestation, the applicability profile). A self-attestation is a formal, legally significant declaration — only make it if it is true.
- A third-party assessor may **validate** an attestation; that raises its weight but is still not a NIST certificate.

---

4. Secure software development — NIST SP 800-218 (SSDF)

SSDF organizes secure development into four practice groups: **Prepare the Organization (PO)**, **Protect the Software (PS)**, **Produce Well-Secured Software (PW)**, and **Respond to Vulnerabilities (RV)**. Requirements below apply to every project; project-specific tooling is recorded in the applicability profile.

4.1 Prepare the Organization (PO.1–PO.5)

- **Security requirements (PO.1).** Security requirements are documented (this standard plus each project's standing contract, e.g., a `CLAUDE.md` or equivalent) and treated as first-class alongside functional requirements. Captured as first-class specs, these requirements drive design and tests per **§5 (Spec-Driven Development)** — recommended.
- **Roles and responsibilities (PO.2).** Maintainer/reviewer roles are defined; security ownership is explicit; onboarding includes secure-development orientation.
- **Supporting toolchains (PO.3).** Source control; CI/CD with automated security checks (§6.2); dependency and secret scanning; pinned, integrity-verified dependencies.
- **Security check criteria (PO.4).** Pass/fail release gates are defined (§6.4); a release does not ship with unresolved high/critical findings.
- **Secure development environments (PO.5).** Disk-encrypted developer machines; **no real PHI in development or test** (synthetic or de-identified data only); least-privilege access to repositories and environments.

4.2 Protect the Software (PS.1–PS.3)

- **Protect code from unauthorized access/tampering (PS.1).** Branch protection, required reviews, least-privilege access; no direct commits to the main branch; signed commits where supported.
- **Verify release integrity (PS.2).** Releases are versioned and integrity-verifiable (checksums/signatures); a software bill of materials (SBOM) is generated per release.
- **Archive and protect releases (PS.3).** Each released version, its build inputs, and its SBOM are archived to support incident analysis and reproducibility.

4.3 Produce Well-Secured Software (PW.1, PW.2, PW.4–PW.9)

- **Secure design and threat modeling (PW.1–PW.2).** Each interface and component is threat-modeled; trust boundaries are identified and documented; designs are reviewed against the security requirements before build. The reviewed design artifact and its acceptance criteria are the spec; **§5** describes the recommended clarify/analyze gates applied before build.
- **Reuse well-secured components (PW.4).** Prefer vetted libraries; avoid rolling custom cryptography.
- **Secure coding practices (PW.5).** Mandatory:
- **Input validation** — validate structure and content at every ingress; reject or quarantine malformed input rather than processing it. *(Example: HL7 message validation in MEFOR.)*
- **Parameterized queries only** — no string-built SQL; use an ORM or parameterized statements throughout.
- **Authentication & authorization** — enforce on every action; deny by default; interface mechanisms per §7.4.
- **Web-service interfaces (REST and SOAP)** — authenticate every endpoint; validate and size-limit payloads against a schema; for SOAP/XML, disable external-entity resolution (XXE) and DTD processing; apply rate limiting and timeouts; never expose stack traces or sensitive data in fault responses.
- **File-handler interfaces** — confine reads/writes to configured directories and canonicalize paths (reject `../` traversal and symlink escapes); validate file type and size by content, not extension; use atomic write-then-rename so partial files are never processed; least-privilege storage, never an executable or web-served path; never execute file contents; scan inbound files for malware where feasible; encrypt sensitive files at rest and securely delete after processing per retention.
- **Cryptography** — TLS for all network communication; encryption at rest for sensitive data; approved algorithms and libraries; use FIPS-validated crypto where a deployment requires it.
- **Secrets** — never in code, prompts, or commit history; sourced from environment/secret store; enforced by pre-commit and CI secret scanning.
- **Error handling and logging** — fail closed; never log secrets or sensitive data; produce a tamper-evident, timestamped audit log.
- **Secure build configuration (PW.6).** Reproducible builds; security-relevant build/interpreter and dependency settings fixed in the pipeline.
- **Code review and analysis (PW.7).** Every change is peer-reviewed; static analysis (SAST) and software composition analysis (SCA) run in CI (§6.2). Review also confirms the change conforms to its spec's acceptance criteria — see **§5** (analyze, cross-artifact coverage); recommended.

- **Test executable code (PW.8).** A maintained automated test suite runs on every change; security test cases are included. Tests SHOULD trace to the spec's acceptance criteria (**§5**, executable acceptance criteria) so coverage is mechanical, not prose. Test *quality* — not just presence — is judged per the Code Quality & Anti-Slop Standards: behavior-verifying assertions over mock choreography, with mutation testing as *guidance*. Heed its **anti-metric rule** — **never gate quality on line-coverage % alone** (a gameable slop-hiding place); measure structure and behavior, not a single scoreboard.
- **Secure defaults (PW.9).** Ships secure-by-default (TLS on, encryption on, least-privilege accounts, verbose audit logging); insecure options require explicit, documented opt-in.

4.4 Respond to Vulnerabilities (RV.1–RV.3)

- **Identify on an ongoing basis (RV.1).** Continuous dependency monitoring; a defined intake channel for internally and externally reported issues (§8).
- **Assess, prioritize, remediate (RV.2).** Findings are triaged by severity with target remediation timelines (set per project in the profile); fixes are verified before closure.
- **Root-cause analysis (RV.3).** Significant vulnerabilities receive a root-cause review; systemic causes feed back into this standard.

---

5. Spec-Driven Development

The practices in this section are a **distilled synthesis** drawn from GitHub Spec Kit, AWS Kiro, and BDD / Specification-by-Example — **not adoption of any external tool or standard, and not a certification.** They formalize habits the reference project already practices. **They are recommended (SHOULD), adopted incrementally**; they add no new blocking release gate, and they do not weaken or replace any security requirement in §4 or §6. Where a practice maps to an SSDF practice, that mapping is noted.

5.1 The spec stack (constitution → decisions → requirements → tasks → verification)

Spec-driven development treats the artifacts that describe *what* a change must do and *why* as first-class, versioned, and connected — so design, build, and verification trace back to an agreed specification rather than to memory. The recommended structure is five layers, each mapping to an SSDF practice. The framing here is generic; the concrete MEFOR artifacts that fill each layer are recorded in Appendix A.7.

Layer	What it holds	Maps to
Constitution	The standing, versioned contract of invariants + vocabulary every later artifact honors.	PO.1
Decisions	Architecture decision records with a build-gating lifecycle (proposed → accepted → superseded/rejected).	PW.1–PW.2
Requirements / sequencing	Numbered, ID'd requirement items, cross-referenced by decisions.	PO.1
Tasks	Decomposition of decisions/requirements into work items / lanes / gates.	PW.1–PW.2

Layer	What it holds	Maps to
Verification	Automated checks + human conformance reviews that test/inspect against the spec.	PW.7 (review), PW.8 (test)

A project SHOULD keep these five layers present and connected; the recommended connections are described in §5.3–§5.5.

5.2 EARS acceptance criteria

A change's behavioral acceptance criteria SHOULD be written in **EARS (Easy Approach to Requirements Syntax)** — a small, constrained grammar that turns prose requirements into testable, unambiguous statements. EARS offers five templates:

Template	Form
Ubiquitous (always-on)	THE SYSTEM SHALL `<response>`
Event-driven	WHEN `<trigger>` THE SYSTEM SHALL `<response>`
State-driven	WHILE `<state>` THE SYSTEM SHALL `<response>`
Unwanted behavior	IF `<condition>` THEN THE SYSTEM SHALL `<response>`
Optional feature	WHERE `<feature>` THE SYSTEM SHALL `<response>`

> **Example (event-driven).** WHEN a message fails strict validation, THE SYSTEM SHALL NAK (AR/AE) and record `ERROR` before any ingress row.

This fits the project's existing posture: the standing contract's invariants already read as SHALL-style statements, and each ASVS "Verify that X" requirement restates as an underlying "the system SHALL X" that EARS can express. EARS also maps cleanly onto the *technical* HIPAA safeguards (access control, audit, integrity, transmission security) the software implements — the administrative safeguards (§7.2) are organizational obligations, not behavioral system triggers, so they fall outside EARS's WHEN/WHILE/IF grammar. *(Lineage: AWS Kiro `requirements.md`; distilled, not adopted.)*

5.3 Requirement → design → tasks → test traceability

Each acceptance criterion SHOULD carry an **ID** linked to the test or fixture that exercises it, so coverage is **mechanical, not prose** — "which criteria are untested" becomes computable rather than a judgment call. The reference project already owns the pieces — decisions in ADRs, requirement IDs in the backlog, tasks in release/multisession plans — and the recommendation is to **connect** them: criterion ID → test/fixture link. *(Lineage: AWS Kiro requirements/design/tasks triad; distilled.)*

5.4 Clarify and analyze (lightweight, advisory)

Two lightweight checks are recommended, both explicitly **advisory, not hard gates**:

- **Clarify** — force ambiguity resolution before build, surfacing and answering open questions while they are still cheap to change. The project already has an informal version: the ADR **"To resolve on acceptance"** block.
- **Analyze** — automated cross-artifact consistency/coverage: does every acceptance criterion have a task and a test? does any artifact contradict the constitution's invariants?

These SHOULD be run as lightweight advisory checks. They introduce no new blocking release gate (cf. §6.4). *(Lineage: GitHub Spec Kit `specify → clarify → plan → tasks → analyze → implement`; distilled.)*

5.5 Executable acceptance criteria (living documentation)

BDD / Specification-by-Example expresses acceptance criteria as Given/When/Then scenarios with concrete `(input → expected outcome)` example tables that **execute** as tests — so specification and verification cannot silently drift, and the spec doubles as living documentation. This fits naturally with EARS's WHEN/THEN phrasing, and the HL7 domain (well-defined inputs, well-defined dispositions) is well suited to example-driven verification. The concrete reference-project opportunity (detailed as R2 in Appendix A.7):

> A project's dry-run gate that already replays fixtures through the real graph but asserts only "didn't error" **SHOULD** be upgraded to assert an **expected disposition per fixture** (e.g. `PROCESSED` / `UNROUTED` / `FILTERED` / `ERROR`), turning it into an executable acceptance-criteria check.

(Lineage: BDD / Specification-by-Example; distilled, not BDD-tool adoption.)

5.6 Constitution as a first-class versioned artifact

A standing, versioned ruleset that all downstream artifacts respect is sound practice — it gives design, decisions, and verification a single source of invariants and vocabulary to honor. The reference project already has it: the project's standing contract / constitution (`../CLAUDE.md`). The only addition is that the **analyze** check (§5.4) can verify no artifact violates the constitution's invariants. *(Validates existing practice; nothing external adopted.)*

> **Recommendation pointer.** For the reference project's existing spec stack and three concrete, recommended improvements (R1–R3), see Appendix A.7.

---

6. Security testing and assessment — NIST SP 800-115

Testing follows the methodology of NIST SP 800-115 (*Technical Guide to Information Security Testing and Assessment*): review techniques, target identification and analysis, and target vulnerability validation.

6.1 Testing tiers

Tier	What	Cadence
Automated (in CI/CD)	SAST, SCA/dependency scan, secret scanning, unit/integration security tests	Every commit / build

Tier	What	Cadence
Dynamic	DAST / authenticated testing of the running app	Per release and periodically
Independent review	Third-party source-code review + penetration test per 800-115 — the project's own pre-production gate (§6.3), **not** an ASVS mandate. It covers the **OWASP ASVS 5.0 Level 3** verification, which is source-assisted/hands-on and may be performed internally; the external engagement adds independence + credibility.	Before a production release; after major change; periodically thereafter

6.2 Internal testing (continuous)

- SAST and SCA run automatically; builds fail on new high/critical findings.
- Secret scanning runs pre-commit and in CI; the full git history is kept clean of secrets, credentials, keys, and any sensitive data.
- Security-focused test cases (authn/authz, input validation, error handling) are part of the standard suite.

6.3 OWASP ASVS 5.0 Level 3 — scope

The independent review is scoped to **OWASP ASVS version 5.0.0** (released May 2025), **Level 3**:

- **Version-pinned citation.** Requirements are cited as `v5.0.0-<chapter>.<section>.<requirement>`; identifiers changed substantially from 4.0.x, so the version is always stated.
- **Scale and level model.** ~350 requirements across **17 chapters**. Levels are **cumulative** — L3 includes all of L1 and L2. **MessageFoundry targets Level 3** (defence-in-depth for the highest-assurance contexts), chosen above the usual L2 norm because the engine carries PHI; L1 and L2 form the cumulative baseline and are assessed first.
- **Access required for L3.** L3 is a white-box / hybrid review: the assessor needs source code, developer access, documentation, and an authenticated test instance running **synthetic, non-PHI** data.
- **Documented Security Decisions (new in 5.0).** Each chapter opens with a requirement to document *how* its controls are applied and *why*. This standard, the per-interface threat models (PW.1), and the secure-default baseline (PW.9) serve as that documentation. **Each project documents which chapters are in scope and records exclusions with justification** — see the applicability profile. (Documenting exclusions is itself an ASVS practice.)
- **5.0 modernizations to honor.** Cryptography (V11) reflects current guidance, including post-quantum considerations; authentication and password rules (V6) align with NIST SP 800-63; ASVS 5.0 scopes to **applications and APIs** (host/network infrastructure is the deployer's responsibility, §2).

The 17 chapters (v5.0.0): V1 Encoding and Sanitization · V2 Validation and Business Logic · V3 Web Frontend Security · V4 API and Web Service · V5 File Handling · V6 Authentication · V7 Session Management · V8 Authorization · V9 Self-contained Tokens · V10 OAuth and OIDC · V11 Cryptography · V12 Secure Communication · V13 Configuration · V14 Data Protection · V15 Secure Coding and Architecture · V16 Security Logging and Error Handling · V17 WebRTC.

Per-project chapter applicability (in scope / excluded with justification) is recorded in the applicability profile.

6.4 Release gates

A production release requires: passing automated checks, no unresolved high/critical findings, current independent-review status (or a documented risk acceptance), and updated evidence (§9).

---

7. Security/privacy controls and HIPAA safeguards — NIST SP 800-66 Rev. 2

For deployments that handle PHI, the software implements security and privacy controls — verified against **OWASP ASVS 5.0** (§6.3) — and maps them to the HIPAA Security Rule using NIST SP 800-66 Rev. 2 (*Implementing the HIPAA Security Rule: A Cybersecurity Resource Guide*). *(A non-PHI deployment may scope the HIPAA mapping out; the control areas still apply.)*

7.1 Applied control areas

These control areas summarize the software's security posture. Each is verified through the OWASP ASVS 5.0 chapter(s) noted (§6.3) and produced by the secure-development practices of §4.

Control area	Applied to the software	ASVS 5.0 chapter
Access control	Least-privilege, role-based access; deny by default	V8 Authorization
Authentication	Authenticated access enforced on every action; strong credential handling; interface mechanisms per §7.4	V6 Authentication (V9/V10 when tokens/OAuth are introduced)
Audit & accountability	Tamper-evident, timestamped audit logging; no sensitive data in logs	V16 Security Logging and Error Handling
Communications & data protection	TLS in transit; encryption at rest; trust-boundary enforcement	V11 Cryptography; V12 Secure Communication; V14 Data Protection
System & information integrity	Input validation; flaw remediation (RV); message/data integrity and durability	V1/V2 Validation; V15 Secure Coding and Architecture
Configuration management	Version control, reviewed changes, secure-default configuration, SBOM	V13 Configuration
Contingency	Backup/restore and replay capability *(deployer operates DR in their environment, §2)*	— *(operational; deployer)*
Risk assessment	Vulnerability assessment by the project; deployment risk assessment by the deployer	§4.4 (RV); §7.3
Secure acquisition	Secure development practices (§4) and vetted third-party components	V15; §4

7.2 HIPAA Security Rule safeguard mapping (via 800-66 Rev. 2)

HIPAA safeguard	Representative requirement	Software implementation	ASVS 5.0
Administrative	Security management, risk analysis, workforce/access management	This standard; least-privilege access; *deployer's risk analysis*	V8, V15

HIPAA safeguard	Representative requirement	Software implementation	ASVS 5.0
Physical	Facility and device controls	*Deployer's environment*	(Deployer)
Technical — Access Control	Unique user ID, authentication, automatic logoff	Authenticated, role-based access; session controls	V6, V7, V8
Technical — Audit Controls	Record and examine activity	Tamper-evident, timestamped audit log	V16
Technical — Integrity	Protect data from improper alteration/destruction	Input validation; durable, ordered processing	V1, V2
Technical — Transmission Security	Protect data in transit	TLS for all sensitive transport	V12
(Addressable) Encryption	Encrypt sensitive data at rest and in transit	Encryption at rest and in transit	V11, V14

7.3 Deployment risk assessment

Each deploying organization conducts its own HIPAA Security Risk Assessment of the deployment (mapped to 800-66 Rev. 2). The project supplies evidence (§9) to support it; it does not replace it.

7.4 Interface authentication standard

Integration software authenticates **systems, not people**, on its interfaces. Each connection uses the strongest mechanism the partner system supports, drawn from the hierarchy below; the mechanism, scope, and credential reference for every connection are recorded in its connection definition. (Maps to ASVS V6/V9/V10/V12; HIPAA person-or-entity authentication and transmission security.) *Which mechanisms a given project implements is recorded in its profile.*

Preferred — system-to-system:

- **Mutual TLS (mTLS).** Client-certificate authentication over **TLS 1.2+ (prefer 1.3)** with strong cipher suites; validate the full chain to a trusted CA, check revocation (OCSP/CRL), rotate certificates before expiry. Where tokens are also used, prefer **sender-constrained (mTLS-bound) access tokens** (ASVS 5.0 V10).
- **OAuth 2.0 client-credentials grant.** The default for machine-to-machine API auth. Prefer **asymmetric client authentication (** `private_key_jwt` **)** over shared secrets; issue short-lived, per-connection scoped tokens; validate issuer, audience, expiry, and scope on every request.
- **SMART on FHIR (Backend Services).** For any FHIR REST interface, authenticate using the SMART **Backend Services** profile — OAuth 2.0 client-credentials with a **signed JWT client assertion** and `system/` scopes; validate granted scopes against the requested operation.

Directory / enterprise integration (e.g., Active Directory):

- **Run under a least-privilege service account — preferably a group-Managed Service Account (gMSA)** on Windows/AD — so the password is auto-rotated and never stored in configuration.

- **Use Kerberos / Integrated Windows Authentication**; prefer Kerberos over NTLM (disable NTLM where feasible) with correct SPNs.
- **Authenticate to databases with integrated authentication** (the service account) rather than a stored database password, where supported.
- **Perform directory lookups over LDAPS (LDAP over TLS) only** — never cleartext LDAP; bind with a least-privilege account.
- **Map roles to directory security groups** for centralized RBAC; if human operators authenticate, **federate to the enterprise identity provider (AD FS / Entra ID) via OIDC or SAML** rather than a local user store.

Legacy / interoperability tier *(supported, least-preferred, documented per connection):* HTTP Basic over TLS, per-connection API keys, or SOAP **WS-Security** (UsernameToken or, preferably, X.509 certificate tokens with message-level signing). Always over TLS; credentials vaulted, scoped per connection, and rotated. Used only when a partner system cannot support a preferred mechanism, with the exception recorded.

Across all mechanisms: TLS everywhere (no cleartext sensitive transport); credentials and keys in a secret store, never in code or config; per-connection least privilege; and per-connection IP allowlisting / network segmentation as defense-in-depth.

---

8. Open-source project security

Because the software is developed in the open and adopted by others, the project also maintains:

- **Repository hygiene.** No secrets or sensitive data ever committed; the full history is scanned and kept clean. A clear `LICENSE`.
- **Coordinated vulnerability disclosure.** A published `SECURITY.md` with a private reporting channel and a disclosure timeline; reported issues feed the RV process (§4.4).
- **Signed, verifiable releases.** Release artifacts are signed and accompanied by an SBOM (PS.2), so adopters can verify provenance and integrity.
- **Contribution review.** All external contributions are security-reviewed before merge; signed commits / DCO required; maintainers gate merges; dependency provenance is checked.
- **Adopter guidance.** A deployment/hardening guide so adopters can stand the software up securely and meet their §2 responsibilities.

---

9. Evidence and attestation

The project maintains a current evidence set so any claim is backed:

- This **Secure Development Standards** document and each project's standing contract.
- **SSDF practice evidence** (toolchain configuration, review records, SBOMs, secure-default settings).
- **Test results** — CI security-scan history; the independent **OWASP ASVS 5.0 Level 3** report and re-test results.
- **Per-project applicability profile** (Appendix A and onward).

- A **claims register** recording each published claim, its wording, and the evidence behind it.

Attestation posture. The software is self-attested as NIST SSDF–aligned, tested per NIST SP 800-115, verified against OWASP ASVS 5.0 Level 3, and built to support HIPAA-compliant deployment (controls mapped to NIST SP 800-66 Rev. 2). Third-party validation of the SSDF attestation and the ASVS 5.0 Level 3 assessment raises the weight of these claims. **Attestations are published with releases** so adopters can rely on them; each adopter still performs its own deployment risk assessment (§7.3). None of these is a NIST certificate; displayable certificates (SOC 2, ISO 27001, HITRUST) are a separate, organization-level track.

---

10. References

- NIST SP 800-218, *Secure Software Development Framework (SSDF) v1.1*
- NIST SP 800-115, *Technical Guide to Information Security Testing and Assessment*
- NIST SP 800-66 Rev. 2, *Implementing the HIPAA Security Rule: A Cybersecurity Resource Guide*
- OWASP Application Security Verification Standard (ASVS) v5.0.0 (May 2025), Level 3

Appendix A — Applicability Profile: MessageFoundry (MEFOR)

The first project under this standard. Future projects add their own profile (Appendix B, C, …) using the same headings.

A.1 Project summary

MessageFoundry (MEFOR) is an open-source **HL7 v2.x interface engine** — a candidate alternative to commercial engines (Corepoint, Mirth Connect, Rhapsody, Cloverleaf). It routes and transforms clinical messages between systems.

Technology stack: Python 3.14+, FastAPI/uvicorn, aiosqlite/SQLite (WAL), `python-hl7` / `hl7apy`, a same-origin browser ops console served at `/ui`, PySide6 (the standalone test harness only — the desktop admin console was retired, BACKLOG #103), Windows/PowerShell deployment; MLLP transport with native MLLP-over-TLS (opt-in via cert config — ADR 0002); application-layer AES-256-GCM encryption at rest (database-native where the backend provides it). Durable message store with FIFO/per-key ordering and dead-letter handling.

A.2 Interfaces and surfaces

- **HL7 v2.x over MLLP** (inbound/outbound). **MLLP-over-TLS is built** (opt-in via per-connection cert config; optional client-cert **mTLS** via a trust anchor), with an off-loopback bind guard and a certificate-expiry monitor — ADR 0002 / WP-13b. Loopback-bound by default; IP allowlisting as additional defense-in-depth.
- **REST and SOAP** web-service interfaces — **outbound destinations built** (per-connection bearer / Basic-over-TLS; SOAP WS-Security + XML-DSig per ADR 0015). A **generic inbound HTTP listener is built** (ADR 0023) as the substrate REST/SOAP-in ride on; ADR 0003/0004 framed the original non-HL7 transport + payload-agnostic ingress design.
- **Database** source (inbound poll) and destination — ADR 0003.

- **File-handler interface** (file-drop pickup / output).
- **Browser ops console** served same-origin under `/ui` — the **sole operator console** since the PySide6 desktop client was retired (BACKLOG #103). It is **on by default** at a loopback bind (`[security].serve_web_console` ; ADR 0065, ADR 0143) and carries **write** actions (replay, purge, connection flags), not read-only as at its M1.

A.3 OWASP ASVS 5.0 Level 3 — chapter applicability

#	Chapter (v5.0.0)	In scope	Notes
V1	Encoding and Sanitization	Yes	HL7 input validation, output encoding, parameterized SQL
V2	Validation and Business Logic	Yes	HL7 structural/content validation; routing/business-rule checks
V3	Web Frontend Security	**No**	No browser-delivered UI (PySide6 desktop + APIs); documented exclusion. Re-scope if a web/admin UI is added
V4	API and Web Service	Yes	**Core surface** — the localhost engine API: authn/authz per endpoint, payload size limits, WS-Origin checks. REST/SOAP outbound destinations plus a **generic inbound HTTP listener** (ADR 0023); **XXE/DTD defenses apply when inbound XML / SOAP-IN body parsing is added** — no inbound XML attack surface yet
V5	File Handling	Yes	File-handler interface: path confinement, content validation, atomic write-then-rename, malware scan, encryption at rest
V6	Authentication	Yes	Per §7.4; align to NIST SP 800-63
V7	Session Management	Yes	API/UI session controls, timeout/logout
V8	Authorization	Yes	Role-based, least-privilege, deny-by-default
V9	Self-contained Tokens	**In scope when introduced; currently N/A**	No JWT/JOSE today — sessions are opaque, server-side, revocable tokens. Applies if/when JWT access tokens are added
V10	OAuth and OIDC	**Partly active**	**Outbound** OAuth 2.0 client-credentials + SMART on FHIR Backend Services and a FHIR REST connector are built (ADR 0024/0043 — `transports/smart.py` / `fhir.py`). **Inbound** OAuth/OIDC (engine as OAuth resource server) remains N/A
V11	Cryptography	Yes	At-rest encryption via **application-layer AES-256-GCM on PHI columns** (database-native where the backend provides it); argon2id passwords; approved algorithms; post-quantum awareness
V12	Secure Communication	Yes	TLS / MLLP-over-TLS; mTLS for system-to-system
V13	Configuration	Yes	Secure-by-default; secrets management; SBOM
V14	Data Protection	Yes	PHI minimization, encryption, no PHI in logs

#	Chapter (v5.0.0)	In scope	Notes
V15	Secure Coding and Architecture	Yes	Threat modeling, secure design, vetted components
V16	Security Logging and Error Handling	Yes	Tamper-evident audit log; fail-closed errors; no PHI/secrets in logs
V17	WebRTC	**No**	Not applicable — no WebRTC; documented exclusion

In scope: 12 chapters active today (V1, V2, V4–V8, V11–V16). V10 (OAuth/OIDC) is now partly active — outbound OAuth 2.0 client-credentials / SMART on FHIR are built (ADR 0024); inbound OAuth/OIDC remains N/A. V9 (JWT) is in scope when JWT is introduced — currently N/A. Documented exclusions: V3, V17.

A.4 Interface authentication mechanisms

Recorded honestly against what is **built today** vs **designed-but-deferred** vs **aspirational/ planned** (§9: every claim is backed by evidence). The §7.4 hierarchy is the target; this is MEFOR's current position on it.

Built (in code today):

- **System-to-system (data plane):** per-connection **HTTP Basic over TLS** and **bearer token / API key** (env-vaulted) on REST and SOAP **destinations**; **database** authentication via SQL login, **Windows Integrated (Trusted Connection)**, or **Microsoft Entra**, over an encrypted connection.
- **Outbound machine-to-service auth: OAuth 2.0 client-credentials** with a **signed-JWT client assertion** and the **SMART on FHIR Backend Services** profile (ADR 0024, `transports/smart.py`); a **FHIR REST connector** (`transports/fhir.py`, plus the read-only Handler `fhir_lookup` — ADR 0043); and **SOAP WS-Security** on the SOAP destination — `<wsse:UsernameToken>` + WS-Addressing/Timestamp and **XML-DSig X.509** signing over the WS-*-wrapped envelope (ADR 0015).
- **Transport TLS:** native **API HTTPS/WSS** and **MLLP-over-TLS** (TLS 1.2+, opt-in via cert config), with optional client-certificate **mTLS** (API `tls_client_ca_file`; MLLP `tls_ca_file`), an off-loopback bind guard, and a certificate-expiry monitor — ADR 0002 / WP-13a/13b.
- **Operator strong-auth (control plane):** native **RFC 6238 TOTP MFA** for **local** accounts (ADR 0002 WP-14, built 2026-06-17) — enrolled per user, enforced via `[security].require_mfa` (on by default; `require_mfa_scope` defaults to every local account) and re-verified at the sensitive-operation step-up boundary; AD/Entra users' MFA is delegated to the IdP. Recovery codes are argon2id-hashed; the TOTP secret is store-cipher protected.
- **Operator / directory (control plane, not interface auth): LDAPS** directory bind (certificate-validated; cleartext `ldap://` refused fail-closed), **Kerberos / SPNEGO** Windows SSO, and **AD security-group → role** mapping for RBAC. These authenticate **human operators** to the console/API, not data-plane systems.

Designed but deferred (ADR 0002 — build before off-loopback exposure):

- **Federated SSO for operators (OAuth 2.0 / OIDC / SAML via Entra)** — gets a dedicated federated-SSO ADR when 0.2 design begins; today's operator directory auth is direct LDAPS bind + Kerberos SSO, not federation. (Native TOTP MFA, transport TLS, MLLP-over-TLS, and client-cert mTLS are all **built** — see above.)

Aspirational / planned (not built, no ADR yet):

- *None outstanding.* The items previously listed here — **OAuth 2.0 client-credentials**, **SMART on FHIR (Backend Services)**, and **SOAP WS-Security (UsernameToken / X.509)** — have since been **built** (ADR 0024 / ADR 0015; see the **Built** tier above). Inbound OAuth/OIDC with the engine as a resource server remains out of scope until a JWT/OIDC-bearer inbound is introduced.

> **gMSA** is a **deployment posture** (run the Windows service under a group-Managed Service Account), not an engine protocol — see `docs/SERVICE.md`. It is recommended, not enforced in code.

A.5 Project-specific parameters

- **Remediation SLA windows** (RV.2) — **confirmed 2026-06-12: Critical ≤ 7 days, High ≤ 30 days, Medium ≤ 90 days** (Low: best-effort). Measured from triage; fixes verified before closure; coordinated disclosure after a fix is available. Published in `.github/SECURITY.md`.
- **Applicable control set** — the tailored control baseline is confirmed per deployment with the deploying organization's security lead (the §7.1 control areas apply; deployment-specific tailoring is the deployer's, §2/§7.3).

A.6 Documented deviations

Honest record of where MEFOR's *current* practice differs from the body of the standard, with the compensating control (the standard requires exclusions/deviations be documented, §6.3).

- **Single-maintainer development (PO.2 / PW.7).** The project is solo-maintained today, so the standard's "every change is peer-reviewed" cannot mean a *human second reviewer*. Compensating controls: blocking automated review (bandit/semgrep SAST, pip-audit SCA, gitleaks), AI-assisted review, branch protection + required CI checks, and no direct pushes to `main`. Revisit when a second maintainer joins. **Detailed record:** the AI-assisted-review compensating control — and the full risk-tiered discipline for building with Claude Code — is operationalized in `Secure_AI_Development_Standards.md`, the companion standard that owns and expands this deviation.
- **Independent ASVS-L3 review & DAST (§6.3 / §6.4).** Not yet performed; a **dated risk acceptance** is in force pre-1.0. An independent engagement is planned — it is a **$25,000–$50,000** commitment that the project intends to fund through a grant or sponsorship rather than licence revenue. MessageFoundry is **self-hosted**, so the decision to deploy it beyond loopback, and the assessment that justifies that decision, rest with the **implementing organization**: this standard states what has and has not been independently verified, and does not gate your deployment on it.
- **Federated operator SSO (§7.4).** API/WSS/MLLP **TLS and client-cert mTLS are built** (opt-in via cert config) and **native TOTP MFA for local accounts is built** (ADR 0002 WP-14); the remaining deferred operator-auth item is **federated SSO (OIDC/SAML via Entra)**, held safe by the fail-closed `127.0.0.1` bind guard. Federation gets a dedicated ADR before off-loopback exposure.
- **Mechanical requirement→test traceability (§5.3, recommended).** Not yet enforced: acceptance criteria are not uniformly ID'd and linked to tests, and the dry-run gate asserts only "didn't error" (R2). This is **not a deviation from a hard requirement** — §5 traceability is **recommended (SHOULD)**, adopted incrementally

— but it is recorded here for honesty. Tracked as R1–R3 in §A.7.

ASVS 5.0 L3 deferred items — accepted / deferred (risk accepted **2026-06-16**, refreshed after MFA + admin-defense landed **2026-06-17**; owner: project maintainer). Each is deferred-by-design behind the fail-closed bind guard or off-loopback-conditional, with the compensating controls below. Detail + build triggers: `security/ASVS-FAILS-REMEDIATION-PLAN.md`; per-requirement verdicts: `security/ASVS-L3-ASSESSMENT.md`. Reviewed at each release and on any trigger below. Those are maintainer-internal documents; `SECURITY-DOCS-POLICY.md` explains what is withheld and what you can request.

- **6.3.3 — multi-factor authentication. Satisfied for local accounts** — native RFC 6238 TOTP MFA is **built** (ADR 0002 WP-14, 2026-06-17), enforced via `[security].require_mfa` (on by default, for every local account) at the step-up boundary; **AD/Entra-account MFA is delegated to the IdP** (the supported enterprise path). No longer a deferred Fail. *(Hardware/WebAuthn second factors are now built — browser WebAuthn passkeys as the phishing-resistant second factor at the step-up boundary, ADR 0068 / WP-14b, behind the `[webauthn]` extra.)*
- **4.1.5 — per-message digital signatures on the PHI data plane.** Deferred-by-design. Transport-level security (TLS 1.2+ floor) over a single-tenant on-prem network is the de-facto standard for HL7 v2 interchange; per-message signing is rare in practice and reserved for partners that contractually require it. *Compensating controls:* TLS + trusted on-prem network, no untrusted intermediary on the supported model. *Build trigger:* a partner contract mandating a message-level signature, or an off-prem / shared-tenant / untrusted-intermediary deployment (SOAP XML-DSig per ADR 0015 §4a, or a detached JWS for HL7/JSON). *Design record:* ADR 0018.
- **8.4.2 — multi-layer administrative-interface defense. Built (2026-06-17, ADR 0002)** — WP-14 MFA is wired as a genuine second factor at the step-up boundary, plus a **new-client-IP contextual-risk signal** (`[auth].admin_new_ip_step_up`, default off) layered over deny-by-default RBAC and the fail-closed `127.0.0.1` bind guard. *Residual (delegated):* device-posture assessment is delegated to the deployment (a managed/attested host + an mTLS client cert terminated at the WP-15 reverse proxy), not done in-process.
- **12.1.4 — TLS certificate revocation (OCSP/CRL).** Off-loopback-conditional. *Compensating controls:* native API/WSS + MLLP TLS built with a pinned 1.2+ floor + a cert-expiry monitor; loopback default. *Build trigger:* off-loopback exposure → OCSP-must-staple / CRL, or documented delegation to the org PKI / TLS terminator (ADR 0002).
- **13.3.3 — key material in an HSM/vault.** Deferred-by-design. The store data key is loaded into engine memory for AES-256-GCM; an attacker able to read process memory already implies host compromise on a single-tenant box. *Compensating controls:* machine-bound DPAPI at rest, restricted service account, on-prem network, host volume encryption. *Build trigger:* off-prem / cloud / shared-tenant or a PHI-critical posture/mandate → the pluggable KeyProvider seam (KMS/Vault/HSM envelope decryption, BEYOND WP-BL3-04).
- **16.4.3 — off-box log / audit shipping.** Off-loopback-conditional. *Compensating controls:* the local `audit_log` is append-only, SHA-256 hash-chained, and read-gated; restricted host. *Build trigger:* off-loopback exposure → structured JSON logging + syslog/SIEM forwarding (BEYOND WP-BL3-20).

A.7 Spec-driven development — existing stack and recommendations

MEFOR already operates the five-layer spec stack of §5; the layers exist but are not yet mechanically connected. Recorded honestly below, with three recommended (SHOULD) improvements.

A.7.1 Existing spec stack (state accurately)

Layer	MEFOR artifact	Notes	SSDF
Constitution	`../CLAUDE.md`	Always-loaded standing contract of invariants + vocabulary.	PO.1
Decisions	`adr/` (`docs/adr/*.md`)	Build-gating lifecycle (`README.md` : Proposed = drafted, no code → Accepted = ratified, build may start → Superseded/Rejected; plus `Reserved` number-allocations and `Dropped`). **ADRs numbered through 0105; some numbers are Reserved/Dropped.** The house pattern includes Context / Decision / Options considered / Consequences / "To resolve on acceptance".	PW.1–PW.2
Requirements / sequencing	`BACKLOG.md` + `FEATURE-MAP.md`	Numbered requirement IDs (e.g. #20, #26, #34), cross-referenced by ADRs; each item names its originating review finding.	PO.1
Tasks	`docs/releases/*-PLAN.md` , `docs/releases/MULTISESSION-PLAN-11.md` (current)	Decompose ADRs/backlog into per-worktree lanes, gates, per-window quartet re-check.	PW.1–PW.2
Verification	`messagefoundry check` (`../messagefoundry/checks.py`) + conformance reviews under `security/`	`validate` (required) + `dryrun` (required when `*.hl7` fixtures exist) + advisory ruff/mypy; reviews: `SDS-CONFORMANCE-REVIEW-*.md` , `ASVS-L3-ASSESSMENT.md` .	PW.8 (test); reviews → PW.7

PW.8 (test executable code) is the home of `messagefoundry check` (validate + dryrun); PW.7 (review/analyze) is the home of the conformance reviews and code review — do not collapse the two. The SDS conformance review already cites SSDF practice IDs natively (PS.2, PO.4, PW.1–PW.2), so this mapping is not a retrofit.

A.7.2 Recommendations (all **recommended / SHOULD**, advisory)

- **R1 — EARS "Acceptance Criteria" block on the ADR template.** Each ADR **SHOULD** carry an EARS "Acceptance Criteria" block, each criterion bearing an ID linked to its test or fixture. Doc-only, zero-code; formalizes the existing SHALL-style house register. *(Lineage: AWS Kiro requirements.md; distilled.)*
- **R2 — Make the dry-run gate executable-spec.** `messagefoundry/checks.py` **SHOULD** read an **expected disposition per fixture** (`PROCESSED` / `UNROUTED` / `FILTERED` / `ERROR`) and assert it, upgrading today's "didn't error" dry-run (`_check_dryrun` asserts only not- `ERROR`) into an executable acceptance-criteria check. **Backward-compatible:** a fixture with no declared expectation keeps today's not- `ERROR` semantics. *(Lineage: BDD / Specification-by-Example; distilled.)*

- **R3 — Promote clarify, add analyze.** The ADR **"To resolve on acceptance"** block **SHOULD** be promoted into an explicit **clarify** step (resolve ambiguity before `Accepted`), and an **analyze**-style advisory coverage check **SHOULD** verify that every **Accepted** ADR's acceptance criteria has a linked test, and that no artifact contradicts a `CLAUDE.md` invariant. **Advisory, not a hard gate** — it belongs in the advisory tier alongside `ruff` / `mypy`, not the required `validate` / `dryrun` tier. *(Lineage: GitHub Spec Kit pipeline; distilled.)*

Version history

Version	Date	Change
2.2	July 30, 2026	**Independent-review deviation reframed (§ deviations).** The independent ASVS-L3 review & DAST is no longer stated as a precondition for off-loopback/production exposure. MessageFoundry is self-hosted, so the deployment decision — and the assessment supporting it — belong to the implementing organization; this standard records what has and has not been independently verified rather than gating deployment on it. The engagement remains planned, at an estimated $25,000–$50,000, intended to be grant- or sponsor-funded. No change to the SSDF / ASVS / HIPAA mappings.
2.1	July 29, 2026	**Code-quality companion added.** Cross-linked the new Code Quality & Anti-Slop Standards (evidence-based anti-slop rubric, ISO/IEC 25010): a companion-standards pointer in §1 and a test-*quality* + anti-metric note at PW.8. No change to the SSDF / ASVS / HIPAA mappings or Appendix A.
2.0	June 24, 2026	Restructured baseline around SSDF, spec-driven development, NIST SP 800-115 testing tiers and SP 800-66 Rev. 2 safeguards. Content carried forward unchanged at this baseline. The full prior changelog — MEFOR-specific drafts → genericization (project-agnostic, with an Appendix A applicability profile) → OWASP ASVS 5.0 Level 3 re-target → NIST SP 800-53 removal → §5 Spec-Driven Development addition and the §5–§9 → §6–§10 renumbering — is preserved in git history.